

Understanding the ETL Mind Map

A detailed walkthrough of Extraction, Transformation, and Load concepts

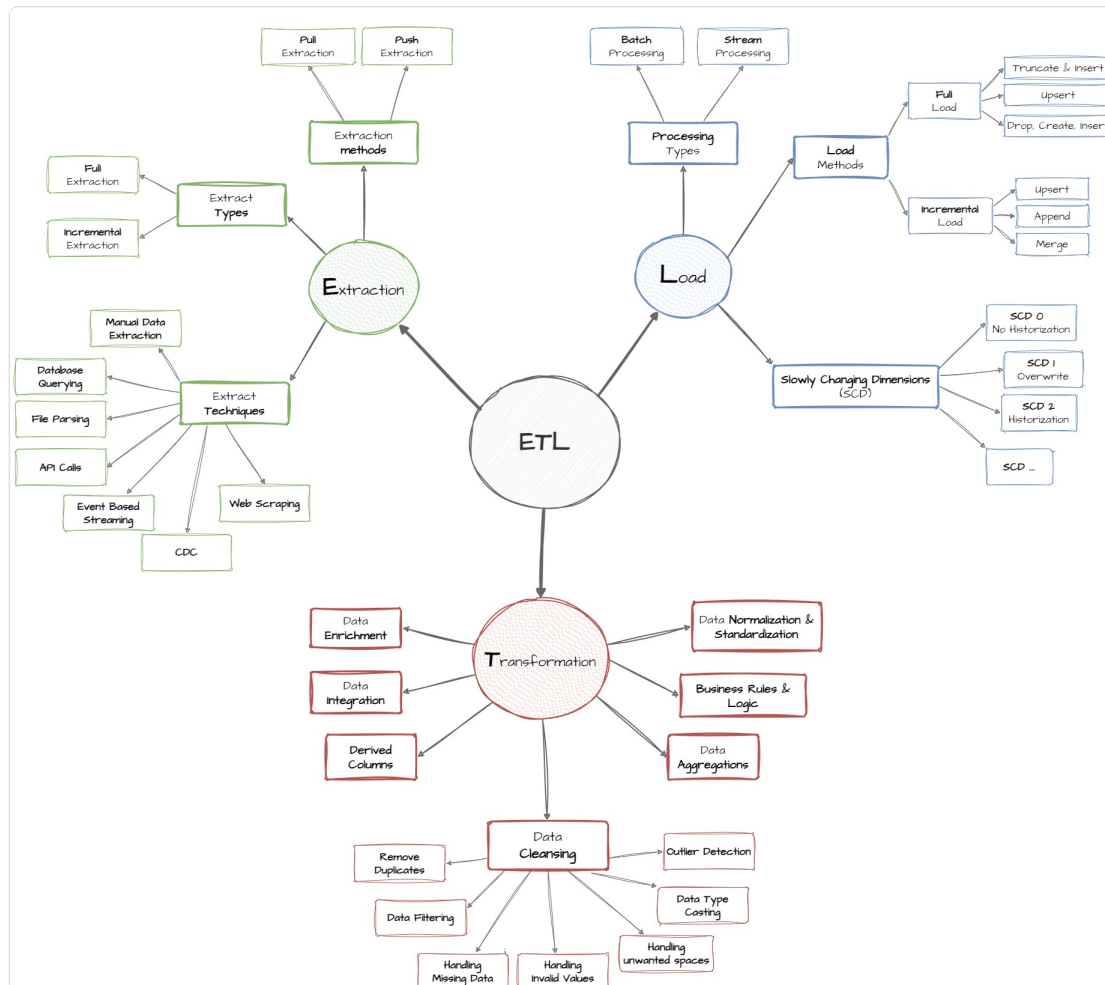


Figure — Full ETL concept mind map: Extraction (green), Transformation (red), Load (blue)

Contents

1. Overview — What is ETL?
2. Extraction — getting data out of source systems
3. Transformation — cleaning, shaping, and enriching data
4. Load — writing data into the target warehouse
5. How the three phases fit together

1 · Overview — What is ETL?

ETL stands for **Extract, Transform, Load**. It is the classic pattern for moving data from one or more source systems into a data warehouse or analytical store, so that it can be queried, reported on, and trusted. The mind map on the cover page breaks the discipline into its three canonical phases — shown as the three large circles labelled **E**xtraction, **T**ransformation and **L**oad — and then expands each phase into the concrete techniques, options, and decisions an engineer has to make when building a pipeline.

The rest of this document walks through every branch and leaf of the mind map, phase by phase, explaining what each term means, why it matters, and how the different options relate to one another.

2 · Extraction

Extraction is the first phase of the pipeline: pulling raw data out of one or more source systems (databases, APIs, files, applications, message queues) before anything is changed or cleaned. The mind map splits Extraction into three sub-branches: **Extraction methods**, **Extract Types**, and **Extract Techniques**.

2.1 Extraction methods

These describe who initiates the movement of data — the source system, or the pipeline itself.

Pull Extraction

The pipeline actively reaches out and requests data from the source — for example, an Airflow task that calls an API or runs a SQL query on a schedule. This is the most common pattern in batch ETL because it gives the pipeline full control over timing and retry logic.

Push Extraction

The source system sends data to the pipeline as soon as it is produced, rather than waiting to be asked — for example, a webhook, a message published to a queue, or a file dropped into a landing bucket that triggers a downstream job. Push extraction suits near-real-time or event-driven architectures.

2.2 Extract Types

These describe how much of the source data is pulled on each run.

Full Extraction

Every run reads the entire dataset from the source, regardless of what changed since the last run. It is simple and safe (nothing can be missed) but becomes expensive and slow as source tables grow, and it re-processes data that hasn't changed.

Incremental Extraction

Only new or changed records since the last successful run are read, typically identified via a timestamp column, an auto-incrementing ID, or a change-log. This is far more efficient at scale but requires the source to expose a reliable way of identifying "what's new," and the pipeline must track a watermark (the last position read).

2.3 Extract Techniques

These are the concrete mechanisms used to actually retrieve the data, regardless of method or type.

Technique	What it means
Manual Data Extraction	A person manually exports or copies data (e.g. downloading a spreadsheet) — low effort to set up, but not repeatable or scalable, and prone to human error.
Database Querying	Running SQL (or similar) queries directly against a source database to retrieve rows, often the fastest and most reliable technique when direct DB access is available.
File Parsing	Reading structured or semi-structured files — CSV, JSON, XML, Parquet — and parsing them into records the pipeline can process.
API Calls	Requesting data from a web service over HTTP(S), typically returning JSON or CSV, as this project does with the Mockaroo API.
Web Scrapping	Extracting data directly from rendered web pages when no API is available, by parsing HTML.
Event Based Streaming	Consuming a continuous stream of events (e.g. Kafka, Kinesis) as they are produced, rather than in scheduled batches.
CDC (Change Data Capture)	Capturing row-level inserts, updates, and deletes directly from a database's transaction log, enabling near-real-time incremental extraction without querying the whole table.

3 · Transformation

Transformation is where raw, extracted data is cleaned, validated, reshaped, and enriched so that it becomes trustworthy and useful for analysis. The mind map shows six top-level activities branching directly off Transformation, plus a large sub-branch dedicated entirely to **Data Cleansing**.

3.1 Data Cleansing

Data cleansing is the set of checks and fixes applied to raw records before they are trusted enough to use. The mind map lists seven specific cleansing activities:

Activity	What it means
Remove Duplicates	Identifying and eliminating records that represent the same real-world event more than once, often by de-duplicating on a natural or business key (e.g. <code>order_id</code>).
Data Filtering	Excluding rows that don't belong in the target dataset — for example, test records, out-of-scope regions, or rows outside a relevant date range.
Handling Missing Data	Deciding what to do when a field is null or absent: drop the row, substitute a default value, or flag it for review.
Handling Invalid Values	Catching values that are technically present but don't make sense (negative quantities, prices of zero, malformed emails) and rejecting, correcting, or flagging them.
Handling unwanted spaces	Trimming leading/trailing whitespace and collapsing inconsistent spacing in text fields, which otherwise breaks joins, deduplication, and grouping.
Data Type Casting	Converting raw values (often strings, since they arrive from CSV/JSON/APIs) into the correct types — dates, integers, decimals, booleans — so the warehouse can store and query them correctly.
Outlier Detection	Identifying statistically unusual values (e.g. an order 1000x the typical size) that may indicate data errors or genuinely exceptional events worth flagging separately.

3.2 Data Enrichment

Adding extra context to a record by combining it with additional reference data — for example, looking up a country name from a country code, or attaching a customer's segment based on their ID. Enrichment increases the analytical value of a record without changing what it fundamentally represents.

3.3 Data Integration

Combining data that originates from multiple, separate source systems into a single, coherent dataset — resolving differences in schema, keys, and formats along the way so that data from different systems can be analyzed together.

3.4 Derived Columns

Creating new columns by computing them from existing ones rather than sourcing them directly — for example, calculating `total_amount` as `quantity × unit_price`. Derived columns encode business calculations once, in the pipeline, so every downstream consumer sees the same consistent value.

3.5 Data Normalization & Standardization

Bringing values into a single, consistent format — standardizing date formats, unit systems, casing of text values, or category labels (e.g. mapping "USA," "U.S.," and "United States" to one canonical value) so that grouping and filtering behave correctly.

3.6 Business Rules & Logic

Encoding the organization's specific definitions of correctness into the pipeline — for example, deciding that a transaction is only "valid" if its quantity and price are positive and its status isn't cancelled or refunded. This is where domain knowledge becomes explicit, testable code rather than an assumption left to each analyst.

3.7 Data Aggregations

Summarizing many detailed rows into fewer, higher-level rows — such as total revenue per country, or average order value per product — typically to power fast, pre-computed reporting tables that don't require re-scanning raw transaction data on every query.

4 · Load

Load is the final phase: writing transformed data into its destination — a data warehouse, data mart, or other analytical store — so it becomes queryable. The mind map breaks Load into **Processing Types**, **Load Methods**, and a dedicated branch for **Slowly Changing Dimensions**.

4.1 Processing Types

Batch Processing

Data is collected and processed in discrete chunks at scheduled intervals (e.g. once a day, as in this project's `@daily` DAG). Batch processing is simpler to reason about and easier to make idempotent, at the cost of latency — the data is only ever as fresh as the last batch.

Stream Processing

Data is processed continuously, record by record (or in very small micro-batches) as it arrives, minimizing the delay between an event happening and it being reflected in the warehouse. Stream processing trades simplicity for low latency and is typically paired with event-based extraction and CDC.

4.2 Load Methods

Full Load

The entire target table is rebuilt or replaced on every run. Three common approaches appear in the mind map:

- **Truncate & Insert** — empty the existing table, then insert all the fresh data.
- **Upsert** — for every incoming row, insert it if it's new or update the matching row if it already exists, even when reloading a full dataset.
- **Drop, Create, Insert** — drop the table entirely, recreate its structure, then insert the data — useful when the schema itself may have changed.

Incremental Load

Only new or changed data is written on each run, leaving unaffected rows untouched. Three approaches appear:

- **Upsert** — insert new rows and update existing ones that have changed, identified by a key.
- **Append** — simply add new rows to the end of the table without touching or checking existing ones; suited to naturally append-only data like raw event logs.
- **Merge** — a more general operation (e.g. SQL's `MERGE / ON CONFLICT`) that can insert, update, or even delete rows in a single, atomic step based on how incoming data compares to what's already there.

4.3 Slowly Changing Dimensions (SCD)

SCDs describe strategies for handling changes to dimension data over time — for example, what happens in the warehouse when a customer changes their address, or a product is renamed. The mind map lists an escalating series of strategies:

Type	Strategy
SCD 0 — No Historization	The dimension value is fixed once loaded and is never updated again, even if the source value changes. Used for attributes that genuinely shouldn't change (e.g. an original order date).
SCD 1 — Overwrite	When a value changes, the old value is simply overwritten with the new one. Simple, but all history of the previous value is lost.
SCD 2 — Historization	When a value changes, a new row is inserted to represent the new version, while the old row is kept and marked as expired (often with effective-date and end-date columns). This preserves full history and allows "as of a point in time" reporting.
SCD ...	The mind map leaves this open-ended because the family continues further in practice — e.g. SCD 3 (keeping only the previous and current value in extra columns), SCD 4 (a separate history table), and hybrid approaches — chosen based on how much history a business actually needs to keep.

5 · How the Three Phases Fit Together

Read as a whole, the mind map describes a single, ordered pipeline. **Extraction** decides how and how much data leaves the source system. **Transformation** decides how that raw data is cleaned, shaped, validated, and enriched until it can be trusted. **Load** decides how the resulting data is physically written into the warehouse, and how history is preserved as that data changes over time.

The choices in each phase interact with one another: an **incremental extraction** naturally pairs with an **incremental load** (upsert, append, or merge), since only a small slice of new data is being handled on each run; a **full extraction** often pairs with a **full load**, since the whole dataset is being refreshed anyway. Similarly, **CDC** extraction is the natural extraction technique to feed a **stream processing** load with **SCD 2** historization, since all three are built around capturing and preserving every individual change as it happens, rather than just the current state.

In the accompanying Sales Data ETL project, these choices are made concretely: **pull-based, full extraction** via API calls from Mockaroo; a rich **transformation** stage covering cleansing, business rules (transaction validity), and aggregation into report tables; and a **batch, incremental load** (append-and-deduplicate on `order_id`, with daily-refreshed aggregates) written to both a DuckDB and a PostgreSQL warehouse in parallel.